

Artigo: Design by Contract

O artigo "Design by Contract", de Bertrand Meyer, surge como um pilar fundamental na busca pela confiabilidade do software, apresentando uma metáfora poderosa: a construção de software como uma sucessão de decisões contratuais documentadas. Esta visão inovadora não apenas ilumina desafios comuns enfrentados por programadores na criação de sistemas corretos e robustos, mas também propõe soluções elegantes e coerentes para aspectos críticos da engenharia de software, com ênfase no paradigma da programação orientada a objetos. Meyer inicia destacando a urgência da questão da confiabilidade, contrastando a escassez de técnicas teóricas no desenvolvimento prático com a crescente relevância da programação orientada a objetos – frequentemente descrita como a "sine qua non" para futuros avanços. Contudo, ele aponta que essa literatura muitas vezes silencia sobre correção e robustez, um paradoxo notável dada a centralidade da reutilização de componentes, que exige intrinsecamente um alto grau de correção.

No cerne da teoria está a noção de contrato, inicialmente explorada através da decomposição de tarefas. Meyer argumenta que, ao delegar sub-tarefas (o "contratado"), é imperativo que o acordo seja precisamente documentado, tal qual em um contrato do mundo real, que protege ambas as partes. Este contrato protege tanto o "cliente" (o chamador da rotina) ao especificar o resultado esperado, quanto o "contratado" (a rotina em si) ao delimitar suas responsabilidades. Ele garante que as obrigações e benefícios sejam explícitos, e a ausência de "cláusulas ocultas" (o contratado não é responsável por tarefas fora do escopo definido) assegura que cada parte sabe exatamente o que esperar e o que entregar.

A materialização desses contratos no código é feita através das asserções, um conceito central na linguagem Eiffel, desenvolvida por Meyer. Pré-condições (*requires*) especificam o que o cliente deve garantir antes de uma chamada de rotina; pós-condições (*ensure*) descrevem o que a rotina assegurará após sua execução; e invariantes de classe (*invariant*) definem propriedades que devem ser verdadeiras para todas as instâncias de uma classe em estados observáveis (antes e depois de cada chamada de rotina pública). O exemplo clássico da rotina `put` em uma tabela ilustra como essas asserções explicitam o comportamento esperado: `count < capacity` como pré-condição, e `item (key) = element; count = old count + 1` como pós-condição, utilizando o operador `old` para referenciar o valor de uma propriedade na entrada da rotina. Essas cláusulas são opcionais, mas sua presença ou ausência tem implicações claras, com `true` sendo a asserção menos restritiva.

Um dos pontos mais contundentes do artigo é a crítica à "programação defensiva". Meyer argumenta que a prática de inserir inúmeras verificações redundantes para lidar com todos os casos possíveis em cada rotina não só aumenta a complexidade do software, mas paradoxalmente diminui sua confiabilidade. Em vez disso, a teoria do contrato propõe que a responsabilidade pela consistência seja explicitamente atribuída: ou o cliente garante a pré-condição, ou a rotina assume a responsabilidade por um caso que não viola

a pré-condição. As asserções, portanto, servem para documentar e depurar – identificar *bugs*, e não para contornar falhas esperadas. O monitoramento de asserções em tempo de execução, configurável em vários níveis (apenas pré-condições, pré/pós-condições, ou todas as asserções incluindo invariantes), é crucial para a detecção de erros durante o desenvolvimento, embora um sistema "correto" teoricamente nunca devesse violá-las – um "paradoxo da semântica das asserções" que ressalta seu papel como ferramenta de *debugging*.

A teoria do contrato também oferece uma estrutura rigorosa para o tratamento de situações anormais e exceções. Meyer critica os mecanismos tradicionais de exceção, como os de Ada, por sua ambiguidade e pela forma como permitem que rotinas "desonestas" falhem sem notificar adequadamente o chamador, potencialmente levando a estados inconsistentes. O exemplo da função `sqr t` que imprime um erro e retorna sem indicação de falha ao chamador, ou a rotina `pop` de uma pilha vazia, são apresentados como demonstrações da falta de um mecanismo que garanta que a falha seja explicitamente comunicada, prejudicando a capacidade do cliente de reagir adequadamente. Em contraste, ele formula três leis de tratamento de exceções: (1) uma rotina deve ou cumprir seu contrato ou falhar; (2) a falha de uma rotina deve *sempre* causar uma exceção no chamador; e (3) a resposta a uma exceção deve ser ou a "retomada" (`re try`), tentando uma estratégia alternativa, ou o "pânico organizado", que implica em restaurar o objeto a um estado consistente (respeitando o invariante de classe) e reportar a falha ao chamador. A linguagem Eiffel implementa isso com as cláusulas `rescue` e `retry`, com `default_rescue` na classe `ANY` permitindo que classes ancestrais definam um comportamento padrão para restauração do invariante, demonstrando uma abordagem mais disciplinada e segura.

Finalmente, a abordagem de Meyer estende-se à programação orientada a objetos, particularmente à herança e ligação dinâmica. A redefinição de rotinas em classes descendentes é reinterpretada como "subcontratação honesta". O "subcontratado" (a versão redefinida da rotina) não pode exigir mais do "cliente" (ter uma pré-condição mais forte que a original) nem entregar menos (ter uma pós-condição mais fraca). Isso se traduz na "regra de redefinição": a pré-condição de uma rotina redefinida deve ser *mais fraca* (ou igual) à original, e a pós-condição deve ser *mais forte* (ou igual). Por exemplo, uma pré-condição `count < capacity` pode ser enfraquecida para `count <= 11 * capacity` se o descendente implementar redimensionamento. Essa regra, implementada em Eiffel com `require` `else` e `ensure` `then`, garante a consistência e a robustez em hierarquias de herança, evitando os "horrores da ligação estática" que poderiam levar a objetos em estados inconsistentes. Para a documentação, ferramentas como `short` e `flat` em Eiffel garantem que a interface de uma classe, incluindo as asserções expandidas de rotinas redefinidas, seja clara e acessível, mesmo em hierarquias complexas.

Em última análise, o "Design by Contract" é um apelo à aceitação de funções parciais na programação. Em vez de buscar um objetivo inatingível de construir sistemas que funcionem sob *todas* as circunstâncias, Meyer defende a criação de componentes de software que declarem explicitamente o que farão e o que não farão. Essa clareza contratual, baseada em asserções precisas, simplifica a arquitetura, melhora a

compreensibilidade e, conseqüentemente, aumenta a robustez e a confiabilidade dos sistemas de software, marcando uma contribuição duradoura para a teoria e prática da engenharia de software.